# RAG: Dynamic Parsing & Dynamic Chunking

**Project Report and Implementation Results**

Prepared by: Shakthi K
Date: 14-10-2026

## 1. Introduction

Retrieval-Augmented Generation (RAG) is a method that helps an AI application answer questions using information retrieved from external documents. Before the AI can use a document, the document must be read, its content extracted, and the content divided into useful chunks.

This project focuses on dynamic parsing and dynamic chunking. Instead of treating every file as one long string or splitting it blindly at a fixed length, the pipeline examines the document structure and creates chunks that preserve useful context.

## 2. Problem Statement

Fixed-size chunking can split asentence, separate a rule from its exception, or detach a table value from its heading. These issues may cause a retrieval system to return incomplete or misleading evidence. The project will compare a simple fixed-size baseline with more structure-aware chunking methods.

## 3. Project Objectives

• Extract text from supported documents such as TXT and PDF, with DOCX support where practical.
• Preserve available headings, page details and source information.
• Implement multiple chunking strategies and inspect their output.
• Retrieve relevant chunks for the same set of test questions.
• Compare retrieval quality using a small evaluation dataset.
• Document the findings and limitations of the prototype.

## 4. Scope and Expected Workflow

The project will be a small Python prototype rather than a complete commercial RAG platform. It should run locally and use sample documents so the demonstration does not depend on a GitHub repository or paid API.

## 5. Dynamic Parsing

Dynamic parsing selects an extraction method based on the input file and the structure found inside it. For example, a text-based PDF can use text extraction, a DOCX file can use a document parser, and an image-only PDF may require OCR. For a short first version, implement reliable text extraction first and clearly report unsupported or low-quality input.

The parser should retain basic metadata such as source filename, page number where available, section heading and content type. This makes it easier to trace a retrieved chunk back to its original document.

## 6. Dynamic Chunking

Dynamic chunking changes chunk boundaries according to the content rather than using one fixed length for every document. Paragraphs under the same heading can be grouped together, while oversized sections can be split at safer boundaries such as paragraph or sentence endings.

- Keep headings connected to the text they describe.
- Avoid empty chunks and unnecessary duplication.
- Apply a maximum chunk size to prevent oversized chunks.
- Preserve table headers with their corresponding row values when tables are supported.
- Attach source and section metadata to every chunk.

## 7. Chunking Strategies to Compare

The same sample documents and questions should be used for each strategy so that the comparison is fair.

**Strategy A – Fixed-Size Chunking**

Split the extracted text into chunks of a selected size, optionally with a small overlap. This is the baseline because it is easy to implement. Its limitation is that a split may occur in the middle of a sentence or separate related details.

**Strategy B – Sentence-Based Chunking**

Split the text into sentences and group neighbouring sentences until the target size is reached. This helps keep complete sentences together, although headings or relationships across paragraphs may still be lost.

**Strategy C – Structure-Aware / Recursive Chunking**

Prefer document structure such as headings and paragraphs. If a section is too large, recursively split it using smaller boundaries while respecting a maximum size. This is intended to preserve context better than arbitrary splitting.

## 8. Embeddings and Retrieval

An embedding model converts each chunk into a numerical vector that represents its semantic meaning. When a user asks a question, the question is embedded using the same model and compared with the stored chunk vectors. The system returns the top matching chunks based on similarity.

For a local demonstration, a lightweight Sentence Transformers model and FAISS or NumPy cosine similarity can be used. No paid LLM API is required just to compare retrieval results.

## 9. Evaluation Method

**Evaluation steps**

- Create atleast fivequestions from the sample documents.
- Record which chunks each strategy retrieves for each question.
- Check whether the expected evidence appears in the top three results.
- Use Recall@3 where relevant expected chunks can be labelled.
- Compare context completeness, relevance, duplicate results and processing time.

## 10. Sample Demonstration

**Sample document:** Employee leave policy.

**Question:** “What should an employee do if they cannot access their work email during password recovery?” **Expected behaviour:** The system should retrieve the recovery exception and retain enough section context to explain the correct process. Compare whether each chunking strategy retrieves the complete rule rather than only a partial sentence.

## 11. Expected Output

• A runnablelocal Python application.
• A view of parsed text and chunks generated by each strategy.
• Top retrieved chunks and similarity scores for each test question.
• A comparison of retrieval results, including Recall@3 if labelled data is available. • A short summary of which strategy worked best on the test documents and why.

## 12. Proposed Technology Stack

**Language:**Python.
• PDF parsing: PyMuPDF or pypdf.
• DOCX parsing: python-docx.
• Chunking: Custom Python functions or LangChain text splitters.
• Embeddings: Sentence Transformers with a lightweight model.
• Vector retrieval: FAISS or NumPy cosine similarity.
• Interface: Streamlit for a simple local UI, or a command-line interface for the fastest version.

## 13. Suggested Project Structure

app.py — simple interface or application entry point

parser.py — document loading and text extraction

chunkers.py — fixed, sentence and structure-aware chunking

retrieval.py — embeddings and top-k retrieval

evaluate.py — sample questions and comparison metrics

sample_docs/ — sample policy and documentation files

requirements.txt — Python dependencies

README.md — setup and usage instructions

## 14. Testing Plan

• Confirm that each supported sample document is parsed.

• Inspect chunk boundaries for missing text and broken sentences. • Confirm that every chunk retains its source metadata.

• Run the same questions against all strategies.

• Check top-k retrieval and evaluation calculations.

• Test the application with an empty query and an unsupported file.

## 15. Limitations

The prototype will be evaluated on a small sample corpus, so the results may not generalize to every document type. Complex tables, scanned documents and OCR may require extra work. Retrieval quality also depends on the embedding model, query wording and the content of the documents.

## 16. Implementation Plan and Conclusion

### Implementation sequence

- Step 1: Createthe projectfolder and install the required packages.
- Step 2: Add sample documents and implement parsing.
- Step 3: Implement the three chunking strategies.
- Step 4: Generate embeddings and retrieve the top three chunks.
- Step 5: Create test questions and compare retrieval output.
- Step 6: Add a simple interface, run tests and document the results.

## 17. Feasibility

Thisproject is feasible as a small internship prototype because it can be developed using Python and commonly available document-processing and embedding libraries. A basic retrieval comparison does not require a paid generative AI API. The main practical requirements are installing dependencies, downloading the embedding model and preparing a small set of documents and questions.

## 18. Conclusion

Thisprojectdemonstrates that RAG quality depends on document preparation as well as the language model. Dynamic parsing helps preserve the information and structure of the source document, while dynamic chunking creates useful evidence units for retrieval. By comparing fixed-size, sentence-based and structure-aware strategies on the same questions, the project can show the trade-offs using actual results rather than assumptions.

The final deliverable is a local working prototype, its source code, sample documents, evaluation output and a README explaining how to run it.

### Reference basis

Prepared from theproject topic and technical direction in the supplied “RAG Ingestion and Dynamic Chunking Pipeline” reference. This document describes a proposed implementation and expected evaluation; measured results should be added after the code is executed.

Github link: https://github.com/Sam960sak/rag_project